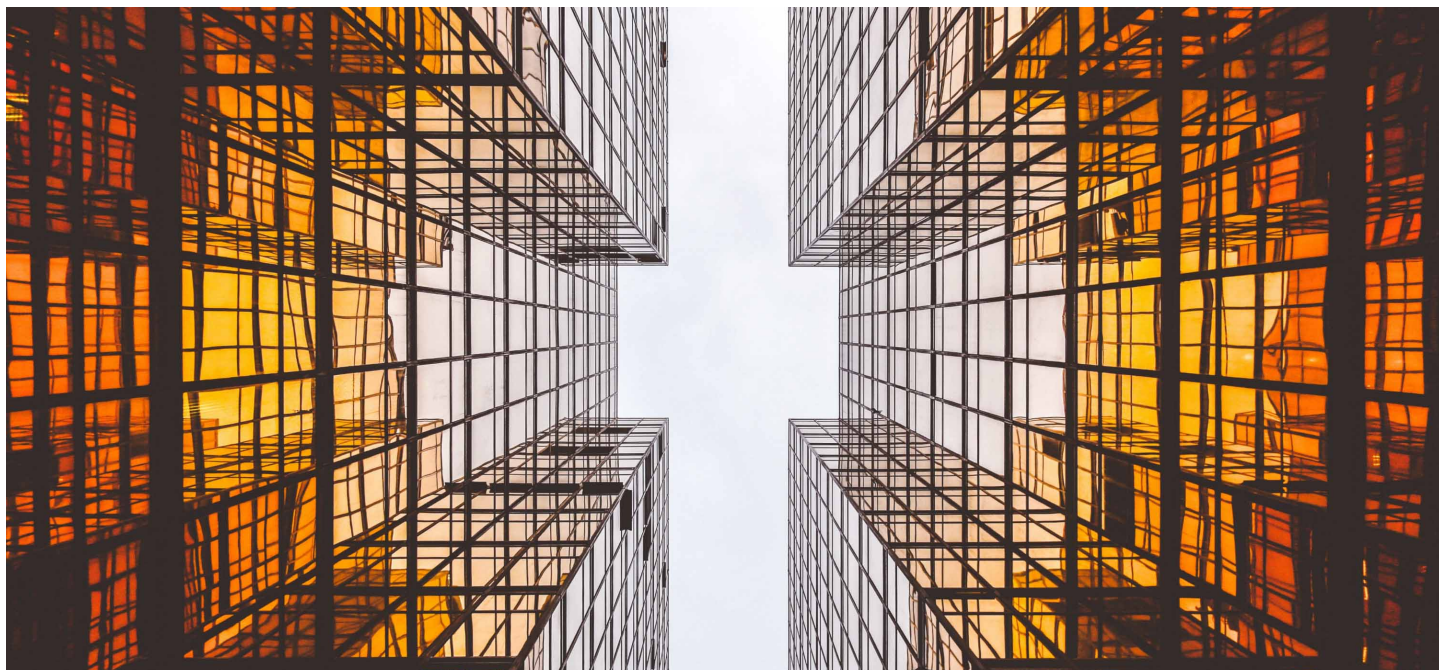




DataFormat for Fine-tuning

1. 垂直领域下微调的重要性？

 通过在领域专属数据上进行微调（Fine-tuning），可以将通用大模型的基础能力转化为专业能力。

通用大模型虽然具备强大的通识与语言理解能力，但在医疗、金融、法律等专业场景中仍存在明显短板：

- **术语混淆**：如医疗领域中「处方药」与「非处方药」的界限可能被错误解释。
- **语义偏差**：金融模型在面对“杠杆”、“流动性”等术语时，往往难以区分语义上下文。
- **逻辑缺陷**：法律问答或文书生成任务中，模型可能忽略条款逻辑、时效性等细节。

Domain-Specific Fine-tuning 成为将通用模型转化为行业智能的关键手段。该方法的核心思路是：

基于通用模型（如 GPT、LLaMA、Qwen、Baichuan），在领域高质量语料（如医疗对话、金融报告、法律案例）上进行增量训练，使模型学习领域内的专用语言模式、逻辑结构与知识体系

2. 微调数据集格式各有所不同？

 需求不同，模型不同，数据格式不同；格式差异并非“错误”，而是“适配不同需求的结果”

同样是对话微调，有的用 instruction-output 结构，有的用 conversations 数组；同样是角色定义，有的写 user，有的写 human。这种差异导致数据难以跨项目复用，往往需要花费大量时间修改格式，甚至因格式不兼容导致训练脚本报错。深究背后原因，主要源于四大核心差异：

- **模型架构差异：**不同模型的设计逻辑决定了其对数据格式的偏好。例如，Llama 系列模型因训练时采用过 Alpaca 格式数据，对 instruction input output 字段的兼容性更高；而 Qwen 模型为优化多轮对话能力，更适配含 role content 的结构化对话格式；BERT 类模型主打文本分类、命名实体识别等任务，仅需简单的 text label 字段即可满足需求。
- **任务类型不同：**任务目标的差异直接影响格式设计。单轮指令微调（如“生成产品说明书”）只需明确“指令 - 输出”对应关系；多轮对话微调（如“客服与用户的多轮交互”）则需记录角色交替顺序；思维链训练（如“数学题解题步骤”）需额外增加推理过程字段；文本分类任务（如“判断客户反馈是否为投诉”）仅需文本与标签的对应。
- **训练框架约定：**不同训练框架对数据格式有默认要求。Hugging Face Transformers 库推荐使用 JSONL 格式，且字段命名需与模型的数据加载器（DataCollator）匹配；若使用自定义训练脚本，开发者可能会根据代码逻辑设计独特字段（如用 question answer 替代 user assistant），导致格式与开源框架不兼容。
- **社区与生态习惯：**开源社区的实践习惯会形成“格式流派”。Alpaca 项目提出的 instruction-input-output 格式因简洁易用，被大量开发者借鉴；OpenAI 推出的 ChatML 格式（含 system user assistant 角色定义）因能精准控制模型行为，成为对话微调的主流选择之一；国内社区围绕 Qwen、ChatGLM 等模型形成的格式规范，也带有明显的本土化实践特征。

3. 微调数据集格式分类详解

3.1 对话角色关键字

- **基础对话角色**
 - system：设定模型身份、行为规则与输出风格（如“你是执业律师，回答需引用《民法典》”）。适用于所有对话类任务，是控制领域专业性的核心字段。
 - user / human：表示用户输入，提供问题、指令或上下文。不同框架命名偏好不同（Alpaca 用 user，部分自研脚本用 human），但语义一致，需在数据集中统一。
 - assistant：记录模型应生成的期望输出，是微调时的学习目标。内容需严格遵循 system 约束，确保专业性与准确性。
- **单轮指令任务字段**
 - instruction：明确任务目标与操作要求（如“分析 2024 年 A 股新能源趋势”），常用于无需模拟用户交互的生成任务。
 - input：可选字段，用于补充背景信息（如“已知装机量增长 20%”），为 instruction 提供执行依据。

- output: 对应 instruction 的期望响应，是单轮任务中的目标输出字段。
- **推理与工具调用字段**
 - think / reasoning: 显式记录分步推理过程（如数学题解题步骤），用于训练思维链（CoT）能力。原生支持 CoT 推理模板的模型。
 - function: 定义模型可调用的工具及其参数（如天气查询接口），需与 function_result 配套使用。
 - function_result: 记录工具调用返回的原始数据，作为生成自然语言响应的依据，不可篡改。
- **非对话类任务字段**
 - text: 待处理的原始文本，用于分类、标注等任务（如用户评论）。
 - label: text 对应的类别或标签（如“负面反馈”），需提前定义统一标准。
 - context: 提供背景文档或参考信息，用于问答抽取任务。
 - question: 明确查询需求，需与 context 中的内容对应。
 - answers: 从 context 中提取的精准答案，支持多答案数组格式。

3.2 各场景（任务需求）Data Format

不同任务类型对数据格式的需求差异显著，可直接作为实际微调中的格式模板参考。

3.2.1 单轮指令微调

单轮指令微调的核心目标是让模型根据单一指令生成对应输出，适用于无需上下文关联的独立任务。其推荐格式基于 3.1 小节的 instruction、input、output 字段设计，采用“指令 - 输入 - 输出”的三层结构，JSON 示例如下：

代码块

```
1  {
2
3    "instruction": "作为金融分析师，分析2024年A股新能源行业趋势",
4
5    "input": "已知2024年新能源装机量同比增长20%，政策端持续加大补贴力度",
6
7    "output": "2024年A股新能源行业呈稳步增长态势：从数据端看，新能源装机量同比增长20%，反映行业需求端旺盛；从政策端看，持续补贴为企业提供了稳定的发展环境，尤其在光伏、储能细分领域，技术迭代与成本下降将进一步推动行业规模扩张。建议重点关注具备核心技术壁垒、产能布局完善的头部企业。"
8
9  }
```

- instruction 字段明确任务目标与模型角色，直接传递“分析行业趋势”的核心需求；

- input 字段补充任务所需的背景数据（装机量增长、政策补贴），为 instruction 提供执行依据；
- output 字段记录模型需学习的专业响应，与 instruction 形成 “需求 - 结果” 的直接对应。

此类格式的典型场景包括领域文本生成（如撰写产品说明书、行业研究报告）、专业翻译（如将英文医疗文献译为中文并规范术语）、信息摘要（如提炼法律条文的核心要点）等。设计逻辑上，通过 instruction+input 明确任务边界与依据，用 output 定义学习目标，让模型快速捕捉 “任务要求 - 输入信息 - 输出目标” 的对应关系，适配单轮无上下文的任务特性。

.....

3.2.2 多轮对话微调

多轮对话微调需模拟真实场景中角色交替交互的过程，核心是记录完整的对话上下文，确保模型理解多轮信息的关联性。推荐格式基于 3.1 小节的 system、user、assistant 字段设计，采用 “对话数组 + 角色字段” 的结构化格式，JSON 示例如下：

代码块

```
1  {
2    "conversations": [
3      {
4        "role": "system",
5        "content": "你是电商平台客服，需耐心解答用户订单问题，优先提供具体解决方案，避免模
        糊回复"
6      },
7      {
8        "role": "user",
9        "content": "我的订单下单后3天还没发货，订单号是123456，麻烦查一下怎么回事？"
10     },
11     {
12       "role": "assistant",
13       "content": "您好，已查询到您的订单（123456）对应商品为预售款，预售周期为7天，当前
        处于备货阶段，预计2天后发出。若您急需使用，可申请更换同品类现货商品，我可为您提供现货链
        接；若选择等待，发货后会通过短信告知物流单号。"
14     },
15     {
16       "role": "user",
17       "content": "那我选择等待，发货后一定要及时通知我哦"
18     },
19     {
20       "role": "assistant",
21       "content": "好的，已为您备注“发货后优先短信通知”，您也可通过APP“我的订单-物流跟
        踪”实时查看进度，有任何问题可随时联系我~"
22     }
23   ]
24 }
```


- conversations 为数组字段，每个元素包含 role（角色）与 content（内容）。system 放在数组首位，提前设定客服角色的服务规则与风格，确保模型初始行为符合领域要求；
- user 字段模拟用户的真实需求，传递订单查询、确认等待等交互指令；
- assistant 字段记录模型的专业响应，与 user 的需求形成“提问 - 解答”的对应关系，且需贴合 system 设定的规则（如提供具体解决方案、避免模糊回复）。

典型场景涵盖电商客服对话、医疗问诊交互（如患者与医生的多轮病情沟通）、技术支持沟通（如用户与工程师的设备故障排查对话）等。设计逻辑上，通过数组记录对话时序，用 system+user+assistant 的角色组合明确交互主体与规则，让模型学习“上下文理解 - 针对性响应”的能力，避免因信息断层导致回复偏离主题。

3.2.3 思维链训练

思维链训练的核心是让模型学习“分步推理”能力，适用于复杂逻辑任务，需额外记录显式的推理过程。推荐格式基于 3.1 小节的 instruction、think、output 字段设计，在“指令 - 输出”基础上新增 think 字段，JSON 示例如下：

代码块

```
1  {
2    "instruction": "作为数学老师，用小学五年级知识解答应用题：某超市搞促销，3瓶饮料售价15元，买5瓶送1瓶。小明带了45元，最多能买到多少瓶饮料？",
3    "think": "第一步，计算单瓶饮料原价：已知3瓶15元，单瓶价格=15÷3=5元；第二步，确定45元可购买的基础瓶数：总金额÷单瓶价格=45÷5=9瓶；第三步，计算赠送瓶数：活动规则“买5送1”，9瓶中包含1个“5瓶”（可送1瓶），剩余4瓶不满足赠送条件；第四步，计算总瓶数：基础瓶数+赠送瓶数=9+1=10瓶；第五步，验证结果：10瓶中，9瓶为购买，1瓶为赠送，总花费=9×5=45元，符合预算。",
4    "output": "小明最多能买到10瓶饮料，具体计算过程如下：1. 单瓶饮料价格：15元÷3瓶=5元/瓶；2. 45元可购买基础瓶数：45元÷5元/瓶=9瓶；3. 赠送瓶数：满足“买5送1”，9瓶可获赠1瓶；4. 总瓶数：9瓶+1瓶=10瓶。最终花费45元，符合预算。"
5  }
```

- instruction 字段明确任务目标（解答数学题）与模型角色（数学老师），传递“用小学五年级知识”的约束条件；
- think 字段按逻辑顺序拆解推理步骤，每个步骤明确“计算依据 - 操作过程 - 中间结果”，为模型提供“如何思考”的方法论，确保推理链路可追溯；
- output 字段基于 think 过程，用更简洁的语言呈现最终结论与核心步骤，保持与推理过程的逻辑一致性，避免结果与思考脱节。

需注意的是，think 字段仅支持具备推理能力的模型（如 Llama 3、Qwen-7B-Chat），不支持推理的模型（如 BERT、早期 GPT-3.5）无需添加，添加后也无法被解析。

典型场景包括数学题解题（如小学应用题、初中几何证明）、逻辑推理分析（如判断合同条款是否符合法律规定）、故障排查（如分析工业设备报错代码的原因）等。设计逻辑上，通过 think 字段向模型

传递推理方法，用 instruction 与 output 固定 “任务 - 结果” 边界，让模型不仅能输出正确答案，还能理解答案的推导过程，提升复杂任务的可解释性与准确性。

3.2.4 文本分类

文本分类任务的目标是让模型学习 “文本 - 标签” 的映射关系，无需角色交互，格式需极简以降低模型学习负担。推荐格式基于 3.1 小节的 text、label 字段设计，采用 “文本 - 标签” 二元结构，JSON 示例如下：

代码块

```
1  {
2    "text": "这款扫地机器人续航只有20分钟，打扫一半就没电了，而且吸不干净毛发，太失望了",
3    "label": "负面反馈"
4  }
```

- text 字段为待分类的原始文本，需完整保留用户反馈、文档片段等原始信息，避免人工修改导致语义偏差（如示例中保留 “续航只有 20 分钟” “吸不干净毛发” 等具体负面描述，确保模型捕捉关键评价点）；
- label 字段为文本对应的类别标签，需提前制定统一的标签体系——以客户反馈分类为例，可分为 “正面反馈” “负面反馈” “中性反馈” 三类，且标签需与文本语义严格匹配（如含 “失望” “没电” 等负面词汇的文本对应 “负面反馈”，含 “很快” “干净” 等正面词汇的文本对应 “正面反馈”）。

需注意的是，文本分类场景中无需添加 system、user 等角色字段，避免冗余信息干扰模型对 “文本-标签” 映射关系的学习。同时，text 字段需经过预处理：去重（删除完全重复的反馈）、纠错（修正 “吸不净” “续行” 等错别字为 “吸不干净” “续航”）、过滤低质内容（如仅含 “好” “差” 等无意义文本），确保数据质量。

典型场景除客户反馈情感分析外，还包括用户意图识别（如将客服对话文本分为 “订单咨询” “售后维权” “产品推荐” 三类）、文档分类（如将法律文本分为 “合同文件” “法条汇编” “案例分析”）、垃圾邮件识别（如将邮件文本分为 “垃圾邮件” “正常邮件”）等。设计逻辑上，通过极简的 “text+label” 结构，让模型专注于文本内容与类别标签的关联学习，减少字段干扰，提升分类效率与准确率。

.....

3.2.5 问答抽取

问答抽取任务需模型从指定背景文档（context）中提取与问题（question）匹配的精准答案（answers），核心是明确 “背景-问题-答案” 的三者对应关系：

代码块

```
1  {
```

```
2   "context": "《儿童常见疾病用药指南》中关于儿童发烧用药的说明：儿童体温超过38.5℃时，可  
使用对乙酰氨基酚或布洛芬混悬液；对乙酰氨基酚适用年龄为3个月以上，单次剂量按体重计算为10-  
15mg/kg，每4-6小时可重复使用1次，24小时不超过4次；布洛芬适用年龄为6个月以上，单次剂量为  
5-10mg/kg，每6-8小时重复使用1次，24小时不超过4次。",  
3   "question": "3个月以上儿童发烧超过38.5℃，可使用的对乙酰氨基酚单次剂量范围是多少？",  
4   "answers": ["按体重计算为10-15mg/kg"]  
5 }
```

- context 字段需提供完整、准确的背景文档，作为答案提取的唯一依据（如示例中完整引用《儿童用药指南》条款），不可遗漏关键信息（如用药剂量），避免模型因信息缺失无法提取正确答案；
- question 字段需清晰明确地传递查询需求，避免歧义——例如不可用“这个药的剂量是多少”，需补充“3个月以上儿童用对乙酰氨基酚的单次剂量”，确保模型精准定位 context 中的答案位置；
- answers 字段：需从 context 中直接提取，不可添加额外信息（如示例中“10-15mg/kg”直接来自指南原文，未新增“建议按体重计算”等额外描述）；多答案时需用数组格式存储，且每个答案需与 question 的提问点一一对应。

若需强化垂直领域约束（如医疗问答需符合临床规范），可在格式中新增 system 字段，例如：

代码块

```
1  {  
2    "system": "你是医疗问答助手，需从参考文档（context）中提取答案，不可编造信息，答案需符  
    合临床用药规范",  
3    "context": "《儿童常见疾病用药指南》...",  
4    "question": "3个月以上儿童发烧超过38.5℃，可使用的对乙酰氨基酚单次剂量范围是多少？",  
5    "answers": ["按体重计算为10-15mg/kg"]  
6  }
```

但需注意，system 字段仅用于约束模型行为，不可替代 context 提供答案依据，避免模型忽略背景文档直接生成答案。

典型场景包括文档阅读理解（如从产品说明书中提取“某型号手机电池容量”）、知识检索（如从法条中提取“劳动合同试用期最长时长”）、医疗问答（如从病历模板中提取“成人高血压诊断标准”）等。设计逻辑上，通过 context 固定答案来源、question 明确查询目标、answers 定义正确结果，让模型学习“定位问题 - 检索信息 - 提取答案”的流程，确保输出答案的准确性与来源可追溯。

.....

3.2.6 工具调用微调

工具调用微调需模型学习“根据用户需求调用工具→获取工具返回结果→生成自然语言响应”的完整流程，JSON 示例如下：

```

1  1. 工具调用
2      "conversations": [
3      {
4          "role": "system",
5          "content": "你可调用“天气查询工具”获取实时天气，工具参数要求：城市（必填，字符串，
如“北京”）、日期（可选，默认当天，格式“YYYY-MM-DD”）；调用后需根据工具返回结果
（function_result）生成自然语言响应，不可直接输出工具参数，需用用户易懂的语言描述天气情
况。"
6      },
7      {
8          "role": "user",
9          "content": "想知道上海2024年10月5日的天气，需要带伞吗？"
10     },
11     {
12         "role": "function",
13         "content": {
14             "name": "weather_query",
15             "parameters": {
16                 "city": "上海",
17                 "date": "2024-10-05"
18             }
19         }
20     },
21     {
22         "role": "function_result",
23         "content": "上海2024-10-05天气：阴转小雨，气温18-22℃，东北风2-3级，相对湿度
85%，降水概率60%。"
24     },
25     {
26         "role": "assistant",
27         "content": "上海2024年10月5日天气为阴转小雨，气温在18-22℃之间，伴有东北风2-3
级，空气相对湿度85%，降水概率60%，建议携带雨伞，出行时注意防滑；若需长时间户外活动，可搭配
薄外套，避免着凉。"
28     }
29 ]
30 }

```

- system 字段：明确工具调用权限（如“可调用天气查询工具”）、参数要求（如必填项“城市”、格式要求）、响应规则（如“不可直接输出参数，需转化为自然语言”），为模型提供工具使用的“操作手册”；
- user 字段：传递用户的实际需求（如“上海 10 月 5 日天气”“是否带伞”），模型需从需求中提取工具调用所需的关键信息（城市“上海”、日期“2024-10-05”）；

- **function** 字段：记录工具调用的具体信息 —— **name** 为工具名称（需与实际工具接口名称一致，如“weather_query”），**parameters** 为工具所需参数（需严格匹配参数类型与必填项，不可遗漏“城市”等必填信息）；
- **function_result** 字段：记录工具返回的原始数据（如天气详情），需完整保留工具输出内容，不可篡改（如不可将“降水概率 60%”改为“可能下雨”），确保模型基于真实数据生成响应；
- **assistant** 字段：模型需将 **function_result** 转化为符合用户需求的自然语言 —— 例如用户问“是否带伞”，需从结果中提取“阴转小雨”“降水概率 60%”等关键信息，明确给出“建议携带雨伞”的结论，而非仅复述工具结果。

此格式适配 Qwen 工具调用版、Llama 3 Tool Use 等模型，若为 OpenAI 生态，需将 **conversations** 替换为 **messages** 数组（参考 3.3.4 小节）。

若工具调用涉及多轮交互（如用户补充“那北京当天呢？”），需延续 **user**→**function**→**function_result**→**assistant** 的字段顺序，确保对话上下文连贯：

代码块

```
1  {
2    "conversations": [
3      // 前序交互（上海天气查询）...
4      {
5        "role": "user",
6        "content": "那北京2024年10月5日的天气怎么样？"
7      },
8      {
9        "role": "function",
10       "content": {
11         "name": "weather_query",
12         "parameters": {
13           "city": "北京",
14           "date": "2024-10-05"
15         }
16       }
17     },
18     {
19       "role": "function_result",
20       "content": "北京2024-10-05天气：晴，气温12-20℃，西北风3-4级，相对湿度40%。"
21     },
22     {
23       "role": "assistant",
24       "content": "北京2024年10月5日天气为晴，气温12-20℃，伴有西北风3-4级，空气干燥（相对湿度40%），无需携带雨伞，建议做好防风措施，可搭配长袖外套应对早晚温差。"
25     }
26   ]
27 }
```

典型场景包括智能助手工具调用（如查询股票行情、预订火车票、翻译文档）、工业设备控制（如调用传感器查询“设备 A 当前温度”、发送指令“启动生产线 B”）、办公自动化（如调用 Excel 工具“统计 Q3 销售额”、调用邮件工具“发送会议通知”）等。设计逻辑上，通过完整记录“需求 - 调用 - 结果 - 响应”链路，让模型学习工具参数提取、结果转化的能力，实现“工具辅助决策”的目标，避免模型因缺乏工具支持无法完成复杂任务（如实时数据查询）。

.....

3.3 各主流模型（Foundation Model） Data Format

模型	官方/社区推荐格式	官方文档链接
LLaMA / Alpaca	<pre>{"instruction": "...", "input": "...", "output": "..."} </pre>	Stanford Alpaca repo
Qwen / ChatGLM	<pre>{"conversations": [{"role": "...", "content": "..."}]} </pre>	Qwen 官方 GitHub
BERT / RoBERTa	<pre>{"text": "...", "label": "..."} </pre>	HuggingFace Transformers Docs
GPT 系列	<pre>{"messages": [{"role": "system/user/assistant", "content": "..."}]} </pre>	OpenAI fine-tuning guide

3.3.1 Llama / Alpaca 生态

字段命名、`apply_chat_template()` 函数存在于 [Hugging Face Transformers 4.40+](#)

格式偏好

该生态以 Llama、Llama 2、Llama 3 系列为代表，早期受 Alpaca 项目影响深远，对单轮指令格式兼容性极强，同时支持多轮对话的 `conversations` 结构。

单轮对话任务

其中，若任务无需额外背景，`input` 可为空字符串；

```
单轮对话任务
1  {
2    "instruction": "将以下英文翻译成中文: Myocardial Infarction",
3    "input": "",
4    "output": "心肌梗死"
5  }
```

多轮对话任务

多轮对话：需使用 `conversations` 数组格式（字段命名与 ChatML 一致，均为 `role/content`，但无 `messages` 顶层数组）：

多轮对话任务

```
1  {
2    "conversations": [
3      {"role": "system", "content": "你是一名医学翻译专家"},
4      {"role": "user", "content": "Myocardial Infarction 是什么? "},
5      {"role": "assistant", "content": "中文名为心肌梗死，是一种急性心血管疾病。"}
6    ]
7  }
```

🌟 关键要求：

- Llama 3 的 tokenizer 原生支持 system/user/assistant 角色标记，强烈建议在多轮任务中显式包含 system 以提升领域控制力；
- 单轮任务中，可将角色约束融入 instruction（如“作为医生，请翻译……”），避免格式冗余；
- 微调时推荐使用 Hugging Face 的 LlamaTokenizer 或 AutoTokenizer，其会自动处理对话模板（chat template）。

🍷 工具支持：

- Hugging Face Transformers 提供 `apply_chat_template()` 方法，可自动将 `conversations` 转为模型可识别的 token 序列；
- 社区广泛使用的 llama-factory、unsloth 等微调框架均原生支持上述两种格式，只需指定 `--dataset_format alpaca` 或 `--dataset_format sharegpt` 即可自动解析。

3.3.2 Qwen / ChatGLM 生态

官方脚本名为 `finetune.py`、`ptuning_v2.py`，并有 `check_data.py` 数据校验工具

该生态以通义千问（Qwen）系列和智谱 AI 的 ChatGLM 系列为代表，专为中文场景优化，强调**结构化对话能力与工具调用支持**。两者均采用基于角色数组的 `conversations` 格式，对 `system` 字段依赖较强，适用于单轮指令、多轮对话及工具调用等多种任务。

👍 格式偏好

单轮对话任务

该生态下，单轮对话任务仍需包装为 `conversations` 数组，不可使用 Alpaca 式的扁平字段。即使仅一轮交互，也必须包含 `system`、`user`、`assistant` 三角色：

代码块

```
1  {
2    "conversations": [
3      {
4        "role": "system",
5        "content": "你是一名金融分析师，回答需基于公开数据，语言简洁专业。"
6      },
7      {
8        "role": "user",
9        "content": "简述2024年A股新能源板块的投资逻辑。"
10     },
11     {
12       "role": "assistant",
13       "content": "2024年A股新能源板块核心逻辑为：政策支持持续、技术迭代加速、全球需求回暖。建议关注具备成本优势与出海能力的龙头企业。"
14     }
15   ]
16 }
```

多轮对话任务

该生态下，多轮对话自然延续 `conversations` 数组，按交互顺序交替添加 `user` 与 `assistant` 消息，`system` 仅出现在首位：

代码块

```
1  {
2    "conversations": [
3      {"role": "system", "content": "你是电商客服，需提供具体解决方案"},
4      {"role": "user", "content": "订单123456三天没发货"},
5      {"role": "assistant", "content": "该订单为预售商品，预计2天后发出....."},
6      {"role": "user", "content": "那我选择等待"},
7      {"role": "assistant", "content": "已为您备注发货后短信通知....."}
8    ]
9  }
```

 关键要求

- 角色顺序严格：`system` 字段必须放在 `conversations` 数组首位，且不可省略（即使无特殊规则，也需填写基础角色定义，如“你是专业助手，回答需准确简洁”），否则模型可能忽略领域约束；

- 工具调用格式统一：Qwen 工具调用版（如 Qwen-7B-ToolCall）需使用 function 与 function_result 字段，且需嵌套在 conversations 数组中；
- 中文字段适配 Qwen/ChatGLM 官方微调脚本，部分版本（如 Qwen2）已支持英文字段解析，若使用 Hugging Face 第三方框架（如 transformers），建议优先用英文字段（role/content），避免解析失败。



工具支持

- 官方提供微调脚本（如 Qwen 的 finetune.py、ChatGLM 的 ptuning_v2.py），直接适配 conversations 格式，可通过参数--data_path 指定数据集路径，脚本自动解析角色字段；
- 提供格式校验工具（如 Qwen 的 check_data.py），可批量检查数据集是否存在角色缺失、顺序错误等问题，输出错误样本列表便于修正。

3.3.3 BERT / RoBERTa 生态

该生态的代表模型包括 BERT-base/-large、RoBERTa-base/-large，主打文本分类、命名实体识别、语义相似度等“非生成类任务”，无需角色交互，格式设计以“简洁高效”为核心，仅需核心任务字段。



格式偏好

文本分类任务：

- 采用 text+label 二元格式，示例如下（情感分析场景）

代码块

```
1  {
2    "text": "这款笔记本电脑续航长达12小时，运行流畅，非常适合办公使用",
3    "label": "正面"
4  }
```

命名实体识别任务：

- 采用 text+entities 格式，entities 记录实体类型与位置，示例如下（金融领域）：

代码块

```
1  {
2    "text": "2024年9月，阿里巴巴集团在杭州举办了云栖大会，发布了最新的人工智能模型“通义千问2.0”",
3    "entities": [
4      {"type": "时间", "value": "2024年9月", "start": 0, "end": 6},
```



```
5      {"type": "企业", "value": "阿里巴巴集团", "start": 8, "end": 14},
6      {"type": "地点", "value": "杭州", "start": 16, "end": 18},
7      {"type": "产品", "value": "通义千问2.0", "start": 28, "end": 36}
8  ]
9  }
```

语义相似度任务

采用 sentence1+sentence2+label 格式，label 表示两句语义相似度（如 0 = 不相似、1 = 相似），示例如下：

代码块

```
1  {
2      "sentence1": "医生建议患者每天运动30分钟",
3      "sentence2": "患者需遵医嘱每日进行半小时体育锻炼",
4      "label": 1
5  }
```



关键要求

- 无角色字段：无需添加 system、user 等对话类字段，避免数据冗余导致模型训练效率下降；
- 文本预处理严格：需对 text/sentence 进行分词（如使用 BERTTokenizer）、截断（长度不超过模型最大输入序列，如 BERT-base 为 512 tokens）、padding 处理，确保输入格式统一；
- 标签编码规范：分类任务的 label 需转化为整数（如“正面 = 0”“负面 = 1”“中性 = 2”），不可直接使用字符串标签，需提供标签映射表（如 label2id.json）供模型解析。



工具支持

- Hugging Face transformers 库提供 DataCollatorWithPadding，可自动对文本进行 padding，适配不同长度的输入；
- 官方提供 run_classification.py 等任务脚本，直接支持 text-label 格式，仅需修改数据加载函数即可适配自定义数据集。

3.3.4 OpenAI 生态（GPT 系列）

该生态的代表模型包括 GPT-3.5-Turbo、GPT-4、GPT-4 Turbo，格式设计以“ChatML”为标准，支持对话、工具调用等多种任务，强调“人与模型的沟通协议”，字段定义严谨且通用。



格式偏好

对话微调任务：

- 使用 messages 数组，角色字段包括 system、user、assistant，示例如下（法律咨询场景）：

代码块

```
1  {
2    "messages": [
3      {
4        "role": "system",
5        "content": "你是法律咨询师，专注于劳动合同纠纷，回答需基于《中华人民共和国劳动合同法》，提供具体法条依据与解决方案"
6      },
7      {
8        "role": "user",
9        "content": "公司未与我签订劳动合同，工作满6个月，我可以主张哪些权益？"
10     },
11     {
12       "role": "assistant",
13       "content": "根据《中华人民共和国劳动合同法》第八十二条，用人单位自用工之日起超过一个月不满一年未与劳动者订立书面劳动合同的，应当向劳动者每月支付二倍的工资。你工作满6个月且未签订劳动合同，可主张以下权益：1. 要求公司支付5个月的二倍工资差额（自用工第2个月起计算）；2. 要求公司补签书面劳动合同；3. 若公司拒绝，可向劳动争议仲裁委员会申请仲裁，提交工资流水、工作记录等证据证明劳动关系。"
14     }
15   ]
16 }
```

工具调用微调任务：

- 新增 function_call 与 function 字段，示例如下（航班查询场景）：

代码块

```
1  {
2    "messages": [
3      {
4        "role": "system",
5        "content": "可调用“航班查询工具”获取航班信息，参数：出发地（必填）、目的地（必填）、日期（必填）、航空公司（可选）"
6      },
7      {
8        "role": "user",
9        "content": "查询2024年10月10日从北京飞往上海的东方航空航班"
10     },
11     {
12       "role": "function_call",
```

```

13     "content": {
14         "name": "flight_query",
15         "parameters": {
16             "departure": "北京",
17             "destination": "上海",
18             "date": "2024-10-10",
19             "airline": "东方航空"
20         }
21     },
22 },
23 {
24     "role": "function",
25     "content": {
26         "name": "flight_query",
27         "parameters": {
28             "departure": "北京",
29             "destination": "上海",
30             "date": "2024-10-10",
31             "airline": "东方航空"
32         },
33     "result": [
34         {"flight_no": "MU5101", "departure_time": "08:00", "arrival_time":
35         "10:30", "status": "正常"},
36         {"flight_no": "MU5103", "departure_time": "14:30", "arrival_time":
37         "17:00", "status": "正常"}
38     ]
39 },
40 {
41     "role": "assistant",
42     "content": "2024年10月10日北京飞往上海的东方航空航班信息如下：1. 航班号MU5101，
43     08:00起飞，10:30抵达，当前状态正常；2. 航班号MU5103，14:30起飞，17:00抵达，当前状态正
44     常。建议提前2小时到达机场办理值机，具体登机口以机场通知为准。"

```



关键要求

- 字段命名固定：必须使用 messages 数组与 role/content 字段，不可自定义（如不可用“dialogs”替代“messages”）；
- system 作用显著：system 字段对模型行为的控制优先级最高，需精准描述角色、规则与输出要求，避免模糊表述（如“回答需简洁”应改为“回答控制在 300 字以内，分点列出核心结论”）；

- 格式兼容性：微调数据集需保存为 JSONL 格式（每行一个 JSON 对象），不可使用 JSON 数组格式，否则无法通过 OpenAI API 上传。

工具支持

- OpenAI 提供 openai-cookbook 代码库，包含格式转换脚本（如将普通对话数据转为 ChatML 格式）、数据校验工具（如检查 JSONL 格式合法性）；
- 通过 openai.fine_tuning.jobs.create 接口上传数据集时，API 会自动校验格式，若存在字段错误，会返回具体错误信息（如 “缺少 assistant 角色” “function_call 字段格式错误”）。

Data Format 选择逻辑

 根据目标模型的技术规范、主流社区的实践习惯，以及自身微调任务的具体目的三者综合权衡

- 务必遵守模型规范：不同模型（如 Llama、Qwen、BERT）对字段命名、角色顺序、输入结构有硬性要求，格式不符将直接导致可能导致训练脚本无法解析或输出异常
- 参考社区规范：遵循 Alpaca、ChatML 等主流格式，可复用成熟的工具链（如数据校验、模板转换、微调脚本），大幅降低工程成本；
- 拟合微调目标：单轮生成、多轮对话、思维链推理、工具调用等任务本质不同，需匹配最能表达任务逻辑的字段组合，避免 “用对话格式做分类” 或 “用扁平结构训多轮”。

- **LoRA: Low-Rank Adaptation of Large Language Models (Hu et al., 2022)**
- **Domain-Specific Fine-Tuning for LLMs, arXiv:2309.01688**
- Hugging Face Blog: *Fine-tuning Large Language Models on Domain Data*

官方链接

[OpenAI Fine-tuning Guide](#)

[Hugging Face Transformers Fine-tuning Docs](#)

[Qwen 官方 finetune 说明](#)

[LLaMA Factory 文档](#)

[Stanford Alpaca Dataset](#)

[ChatGLM 官方文档](#)

